



Université de
Technologie de
Compiègne

LO17/AI31 : Indexation et Recherche d'Information

Rapport de Projet RAG – Cinéma Français

Membres du groupe :

Bouaziz Selima

Lozac'H Mael

Marko-Lafon Clothilde

Peter-Hemon Edeline

Groupe :

D1-G4



Sommaire

1	Introduction	2
1.1	Choix du sujet	2
1.2	Architecture générale	2
2	Description des modules	3
2.1	Module principal (main.py)	3
2.2	Module de crawler (crawler.py)	3
2.3	Module d'extraction (film_parser.py)	3
2.4	Module de base de données SQL (database.py)	4
2.5	Module de retriever (retriever.py)	4
2.6	Module de génération (generator.py)	5
2.6.1	Modèle de génération	5
2.6.2	Prompt système	5
2.6.3	Adaptive RAG	5
2.7	Application Streamlit (app.py)	5
2.8	Synthèse des choix de conception	6
3	Évaluation	8
3.1	Métrologie	8
3.2	Résultats	8
4	Conclusion	9
5	Annexes	10



Partie 1

Introduction

Ce projet consiste à implémenter un système RAG (*Retrieval-Augmented Generation*) sur le sujet de notre choix. Nous avons décidé de nous pencher sur le cinéma, et plus particulièrement le cinéma français, en nous appuyant sur la liste Wikipedia des plus grands succès français au box-office mondial (voir annexe), ainsi que sur les pages Wikipedia dédiées à chacun de ces films.

Pour le modèle de langage, nous avons opté pour **Ollama** afin de pouvoir faire tourner le système entièrement en local, sans dépendance à une API externe. Nous avons suivi le notebook officiel de Gemini (voir annexe) comme base de départ, puis nous avons adapté et enrichi le code pour notre sujet.

1.1 Choix du sujet

azerty

1.2 Architecture générale

Nous avons divisé le code en plusieurs fichiers Python afin de faciliter la compréhension, la maintenance et les tests. Notre structure a évolué au fil du projet et des implémentations conservées. Le système final suit l'arborescence suivante :

```
[main.py]
|
+---> [crawler.py]      -> Récupération des URLs des films
|
+---> [retriever.py]    -> Construction de la base vectorielle (Chroma)
|       |
|       +---> [film_parser.py] -> Extraction du contenu des pages Wikipedia
|
+---> [generator.py]    -> Génération des réponses avec Ollama
|       |
|       +---> [router.py] ->
|
+---> [app.py] -> Application Streamlit
```



Partie 2

Description des modules

2.1 Module principal (main.py)

Ce fichier est le point d'entrée du programme. C'est lui qui gère l'exécution des autres modules dans l'ordre, correspondant aux étapes suivantes :

- **Configuration** : on initialise les paramètres du système (modèle, agent utilisateur).
- **Crawling** : on récupère les URLs des pages Wikipedia de chacun des films via la page initiale qui les liste.
- **Indexation** : on construit la base vectorielle à partir de ces pages. Si elle existe déjà sur le disque, elle est chargée directement pour éviter de tout recalculer.
- **Interaction** : correspond à la boucle de questions/réponses entre le système et l'utilisateur. L'utilisateur pose une question, et le système affiche la réponse générée par le LLM. Cette boucle continue jusqu'à ce que l'utilisateur l'interrompe (Ctrl+C).

2.2 Module de crawler (crawler.py)

Ce fichier comporte deux fonctions :

- **get_movie_titles()** : cette première fonction extrait le titre et l'année de sortie des films depuis la page Wikipedia initiale. Pour cela, elle utilise la librairie BeautifulSoup pour parser le contenu HTML, puis cible les balises `<table class='wikitable'>` et en récupère la deuxième colonne (titre) ainsi que la première colonne (année) de chaque ligne. Elle nettoie ensuite les références de type [1] inutiles avec un regex.
- **load_movie_urls()** : cette seconde fonction convertit chaque titre en URL Wikipedia via l'API Wikipedia. Elle gère plusieurs cas spécifiques pour éviter les erreurs. En cas d'homonymie (`DisambiguationError`), le système tente jusqu'à trois variantes du titre dans l'ordre suivant : d'abord le titre exact, puis titre (film), et enfin titre (film, année) pour les pages dont le nom inclut l'année de sortie (comme *Jeanne d'Arc (film, 1999)*). La fonction gère également les erreurs réseau en attendant quelques secondes avant de continuer, pour respecter les limites de l'API Wikipedia.

Plusieurs choix d'implémentation permettent le bon fonctionnement de ce module. L'utilisation d'un dictionnaire pour dédoubler les titres garantit qu'on ne traite pas deux fois le même film, même s'il apparaît dans les deux tableaux de la page. Le paramètre `auto_suggest=False` évite les suggestions automatiques de Wikipedia qui pourraient mener à des pages non souhaitées.

2.3 Module d'extraction (film_parser.py)

Ce fichier est le cœur de l'extraction des données. C'est lui qui permet de rendre la base de connaissances suffisamment complète pour répondre aux questions de l'utilisateur.

Contrairement au notebook qui automatise ce processus avec `UnstructuredURLLoader`, nous avons fait un parsing personnalisé adapté au format des pages Wikipedia. On analyse la page de chaque film récupérée avec le module précédent, et deux sorties distinctes sont produites selon le type de données. L'élément renvoyé suit le format `tuple[str, dict]` :

- **Le str** contient un texte continu destiné à la recherche vectorielle. Il comprend l'introduction si elle existe, ainsi que les sections synopsis, accueil et analyse.
- **Le dictionnaire** contient des données structurées destinées à être indexées séparément dans Chroma : la fiche technique, la distribution et les distinctions.



Nous avons fait ce choix de séparation afin d'avoir le maximum d'informations utiles pour répondre aux questions, tout en évitant de noyer des données factuelles précises (comme un réalisateur ou un nombre de César) dans des passages narratifs.

Ces éléments sont extraits grâce à BeautifulSoup en parcourant les balises `<h2>` pour organiser le contenu en sections, puis le texte des paragraphes `<p>` et des listes `<ul>` ou `<ol>`. Pour les sections structurées, on stocke temporairement les éléments HTML bruts dans `sql_elems` pour les traiter séparément. Les références Wikipedia et les espaces multiples sont nettoyés par la fonction `clean_wiki_refs()`.

Les distinctions sont organisées par sous-catégories (par exemple [César], [BAFTA]) grâce aux balises `h3/h4`, ce qui permet de conserver le contexte de chaque récompense dans une liste plate.

Ces fonctions sont compatibles avec les deux formats de Wikipedia : l'ancien format avec balises `<h2>` directes, et le nouveau format depuis 2023 avec des `<div class="mw-heading2">`.

2.4 Module de base de données SQL (database.py)

Nous avons initialement tenté de mettre en place une BDD Sqlite3 contenant les informations exactes (fiche technique, distribution, etc.) répartis lors du module d'extraction dans des tables structurées. On attendrait donc du générateur qu'il puisse en premier lieu estimer si la question nécessite une réponse trouvée dans la base de données, dans le dossier `chroma_db`, ou les deux, avant de formuler une requête SQL si nécessaire (le modèle des tables lui étant fourni) et enfin de récupérer la ou les information(s) désirée(s).

Cette fonctionnalité a été initialement mise en place comme décrit ci-haut, avant d'être retirée pour des soucis de performance. En effet, le choix du LLM employé ne s'apprêtait pas à ce type de requêtes, malgré les différentes tentatives d'amélioration et d'accommodation. Suite à cela, les informations exactes ont été traitées comme nous avons pu le voir lors de la description des modules.

2.5 Module de retriever (retriever.py)

Ce fichier construit la base vectorielle qui permet la recherche sémantique. Il suit les étapes suivantes :

- **Vérification de persistance** : si Chroma existe déjà sur le disque et n'est pas vide, on le charge directement, évitant ainsi de tout recalculer à chaque lancement.
- **Extraction** : pour chaque URL, on extrait le texte et les données structurées avec `film_parser.py`.
- **Découpage (*chunking*)** : le texte narratif est découpé en segments de 1000 caractères avec un chevauchement de 200 caractères, pour ne pas couper les informations en plein milieu d'une phrase.
- **Contextual Retrieval** : chaque chunk est préfixé par [Film : <titre>] avant indexation, selon le principe du *Contextual Retrieval*¹ proposé par Anthropic (2024). Cela permet au LLM d'identifier à quel film se rapporte un extrait même sorti de son contexte, et évite les confusions entre films lors de la génération (voir tableau 2.1).
- **Indexation des données structurées** : la fiche technique, la distribution et les distinctions sont converties en documents textuels séparés et indexées avec des métadonnées de type (`fiche_technique`, `distribution`, `distinctions`). Ces métadonnées sont ensuite exploitées par le routeur pour filtrer les chunks selon la nature de la question.
- **Calcul des embeddings et stockage** : les embeddings sont calculés en parallèle par lots de 50 chunks avec 4 threads, puis stockés dans Chroma.

Pour les embeddings, nous avons choisi le modèle **BGE-M3** via Ollama. C'est un modèle multilingue qui offre un meilleur support du français que le modèle `nomic-embed-text` utilisé initialement, avec une dimension de vecteur de 1024 (contre 768). Ce changement nécessite de reconstruire la base Chroma car les dimensions des vecteurs sont incompatibles.

1. <https://www.anthropic.com/news/contextual-retrieval>



2.6 Module de génération (generator.py)

Ce fichier définit la chaîne RAG et les paramètres du modèle de génération.

2.6.1 Modèle de génération

Nous utilisons **Qwen2.5 7B** (qwen2.5:7b) via Ollama comme modèle de génération. Ce modèle offre un bon équilibre entre qualité de réponse et rapidité d'exécution en local. La fenêtre de contexte est fixée à 8192 tokens, ce qui est suffisant pour accueillir les 8 chunks retournés par le retriever (environ 2000–2500 tokens) ainsi que le prompt système.

2.6.2 Prompt système

Le prompt système définit les règles strictes que le modèle doit suivre :

- Répondre uniquement à partir des extraits fournis, sans utiliser de connaissances générales.
- En l'absence d'information, répondre explicitement « *Je ne dispose pas de cette information dans mes sources.* »
- Ne pas mélanger les informations de films différents dans une même affirmation, sauf si la question demande explicitement une comparaison.
- Préciser le titre du film concerné lorsque plusieurs films apparaissent dans les extraits.

2.6.3 Adaptive RAG

Nous avons enrichi le module de génération avec un mécanisme d'**Adaptive RAG** : plutôt que d'utiliser systématiquement les mêmes paramètres de retrieval, le système adapte sa stratégie de recherche selon la nature de la question posée. Cette logique est implémentée dans un fichier `router.py` dédié.

Le routeur classe chaque question dans l'une des trois catégories suivantes à l'aide de règles déterministes basées sur des mots-clés :

- **Factuelle** (ex. « *Qui a réalisé Intouchables ?* ») : le retrieval se limite à $k = 4$ chunks, filtrés sur les types `fiche_technique`, `distribution` et `distinctions`. On évite ainsi de ramener des passages narratifs inutiles pour une question factuelle.
- **Narrative** (ex. « *De quoi parle Les Visiteurs ?* ») : comportement par défaut avec $k = 8$ chunks sur l'ensemble du corpus, sans filtre.
- **Comparaison** (ex. « *Compare Intouchables et Bienvenue chez les Ch'tis* ») : on augmente à $k = 14$ chunks avec une recherche par *Maximum Marginal Relevance* (MMR) pour maximiser la diversité des résultats et s'assurer que les deux films sont bien représentés dans le contexte envoyé au LLM.

Ce mécanisme s'intègre dans la chaîne LangChain via un `RunnableLambda` qui intercepte la question avant le retrieval, choisit la stratégie adaptée, puis passe la main au reste de la chaîne sans modifier le prompt ni le modèle.

2.7 Application Streamlit (app.py)

Ce module fournit une interface graphique simple et moderne, baptisée « Pro'jecteur », permettant de substituer la boucle d'interaction du main par une expérience utilisateur web fluide. L'application s'articule autour de plusieurs mécanismes clés :

- **Mise en cache** : Pour éviter de reconstruire la chaîne RAG et de recharger les structures de données lourdes (ChromaDB) à chaque message, le décorateur `st.cache_resource` est appliqué à la fonction d'initialisation. De plus, un fichier de cache local nommé `movies_list.json` permet à l'utilisateur de connaître les films connus par le RAG.



- **Gestionnaire de l'historique** : L'état de la discussion est maintenu à court terme dans la mémoire de la session via le dictionnaire `st.session_state`. Pour assurer la pérennité des échanges, un système de sérialisation sur disque a été mis en place dans un fichier `conv_history.json`. L'utilisateur a donc accès à un historique fonctionnel sur lequel il peut recharger une ancienne session ou basculer d'une discussion à l'autre.
- **Suggestions de prompts** : L'interface intègre deux dispositifs complémentaires d'aide à la formulation :
 - *Boutons de suggestions* : Placés sur la page d'accueil, ils affichent des questions types pré-définies (ex. « *Qui a réalisé Intouchables ?* ») qui s'injectent directement dans le pipeline de traitement via une variable d'état transitoire `pending_q`.
 - *Explorateur dynamique de films* : Un menu déroulant (`st.sidebar.expander`) liste dynamiquement tous les films indexés. Chaque titre de film est encapsulé dans un bouton stylisé cliquable, qui permet de formuler une requête sur le film en question (ex. « *Parle-moi du film [Titre]* »).

2.8 Synthèse des choix de conception

Le tableau 2.1 reprend, pour chaque module, le principal obstacle rencontré et le choix d'implémentation retenu pour y répondre.

TABLE 2.1 – Synthèse des choix de conception

Module	Obstacle rencontré	Choix retenu
<code>crawler.py</code>	Homonymie Wikipedia (un titre de film correspond aussi à une personne ou une autre œuvre)	Recherche en cascade : Titre (film, année) → Titre (film) → Titre, avec <code>auto_suggest=False</code>
<code>crawler.py</code>	Limite de débit de l'API Wikipedia	Pause périodique toutes les 10 requêtes et gestion des erreurs réseau
<code>film_parser.py</code>	Coexistence de deux formats HTML Wikipedia (<code><h2></code> vs <code><div class="mw-heading2"></code> , ancien/nouveau skin depuis 2023)	Fonctions de détection compatibles avec les deux formats (<code>is_h2</code> , <code>heading_text</code> , <code>flat_elems</code>)
<code>film_parser.py</code>	Mélanger texte narratif et données factuelles nuit à la recherche et à la génération	Sortie double : texte continu pour l'indexation vectorielle + dictionnaire structuré (fiche technique, distribution, distinctions)
<code>database.py</code>	Stocker les données structurées en SQL alourdit le pipeline sans bénéfice pour la recherche sémantique	Abandon de SQLite au profit de documents Chroma typés par métadonnées
<code>retriever.py</code>	Reconstruire l'index vectoriel à chaque lancement est coûteux	Persistance de la base Chroma sur disque, rechargée si non vide
<code>retriever.py</code>	<code>nomic-embed-text</code> (768 dimensions) offre un support limité du français	Passage à <code>bge-m3</code> (1024 dimensions, multilingue), avec reconstruction complète de la base
<code>retriever.py</code>	Un chunk isolé de son film d'origine perd son contexte, causant des confusions entre films	Préfixe <code>[Film : <titre>]</code> ajouté à chaque chunk (Contextual Retrieval)
<code>retriever.py</code>	Calcul séquentiel des embeddings trop lent (environ 90 films)	Calcul par lots de 50 chunks, parallélisé sur 4 threads



Module	Obstacle rencontré	Choix retenu (suite)
generator.py	La recherche MMR ($k = 20$, <code>fetch_k=100</code>) ramène trop de bruit et écarte des chunks pertinents au nom de la diversité	Recherche par similarité simple, $k = 8$ par défaut
generator.py	La fenêtre de contexte par défaut d'Ollama tronque silencieusement le prompt	Fixation explicite de <code>num_ctx=8192</code>
generator.py	Réponses hallucinées ou mélangeant plusieurs films	Prompt système structuré : règles de fidélité, formule de refus fixe, interdiction de mélanger les films
router.py	Une stratégie de retrieval unique ne convient pas à tous les types de questions	Adaptive RAG : classification factuelle / narrative / comparaison, avec k et filtre adaptés ($k = 4$ filtré, $k = 8$, $k = 14$ en MMR)
eval_qa.py	Aucune évaluation formalisée ne permettait de mesurer l'impact des changements	Jeu de questions de référence et scores de précision/rappel (retrieval) et d'exactitude (génération), comparés avant/après

Ce parcours du projet montre que la qualité d'un RAG dépend avant tout de l'adéquation entre l'indexation (découpage, métadonnées, embeddings) et la nature des questions posées, bien plus que du choix du modèle de génération lui-même.



Partie 3

Évaluation

Nous avons mis en place un module d'évaluation indépendant (`eval_qa.py`) pour mesurer les performances du système de manière reproductible. Ce module permet de tester le retriever et la chaîne RAG séparément, et de comparer l'impact des différentes améliorations apportées au fil du projet.

3.1 Métrologie

L'évaluation du retriever repose sur deux métriques calculées sur un jeu de questions de référence :

- **Précision** : proportion de chunks retournés qui sont effectivement pertinents pour la question.
- **Recall** : proportion de chunks pertinents effectivement retrouvés parmi tous les chunks pertinents existants.

Pour la qualité des réponses générées, nous utilisons une évaluation automatique : le LLM est invité à noter la réponse produite par rapport à une réponse de référence, selon plusieurs critères (exactitude, complétude, absence d'hallucination).

3.2 Résultats

Les améliorations successives apportées au projet (passage de `nomic-embed-text` à `bge-m3`, ajout du Contextual Retrieval, implémentation de l'Adaptive RAG) ont été mesurées avec ce module pour valider leur impact sur les performances. Ces résultats sont disponibles dans le fichier `eval_results_run.json`.



Partie 4

Conclusion

Ce projet nous a permis de construire un système RAG complet, en allant au-delà du notebook de départ. Nous avons notamment implémenté un parsing personnalisé adapté au format des pages Wikipedia, un indexage différencié entre données narratives et données structurées, et un mécanisme d'Adaptive RAG pour adapter dynamiquement la stratégie de retrieval selon la nature de la question.

Une piste d'indexation des données structurées sous forme d'une base de données SQL a été envisagée, bien que finalement abandonnée faute de résultats satisfaisants.

On remarque à travers ce projet que la qualité d'un RAG repose en grande partie sur la qualité de l'indexation : un bon découpage, des métadonnées bien choisies et un modèle d'embedding adapté à la langue ont un impact beaucoup plus fort sur les résultats que le choix du modèle de génération lui-même.

Parmi les pistes d'amélioration envisageables, on peut citer l'ajout d'un hybrid search (BM25 + vecteur) pour mieux retrouver les noms propres, un mécanisme de query rewriting pour reformuler les questions avant le retrieval, ou encore un reranking par cross-encoder pour affiner l'ordre des chunks retournés.



Partie 5

Annexes

Le site Wikipedia sur lequel ce projet se base :

https://fr.wikipedia.org/wiki/Liste_des_plus_gros_succ%C3%A8s_fran%C3%A7ais_au_box-office_mondial

Le notebook Gemini :

https://github.com/google-gemini/cookbook/blob/main/examples/langchain/Gemini_LangChain_QA_Chroma_WebLoad.ipynb

Les notebooks suivants nous a été très utiles au fil de ce projet :

<https://github.com/langchain-ai/rag-from-scratch/blob/main/README.md>